

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT
1956

ENATHUR, KANCHIPURAM - 631 561 DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING



NAME :T.venkata gowtham

REG NO :11249A387

CLASS : S4

SUBJECT CODE : BCSD244A12

SUBJECT NAME :DATA SCIENCE TOOLKIT LAB

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT
1956

ENATHUR, KANCHIPURAM - 631 561 DEPARTMENT OF
COMPUTER

SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to cer fy that this is the bonafide record of work done by Mr/Ms.

T Venkata gowtham

with Reg. No 11249A387 done by of II-B.E.(CSE) in –

DATA SCIENCE TOOLKIT LAB for Lab during the academic year 2025
2026.

Sta on : ENATHUR Date :

Staff-in-charge
Department

Head of the

Submitted for the practical examination held on _____

Examine-1

Examine -2

INDEX

Sl.NO	PROGRAM NAME	DATE	PAGE NUM	SIGNATURE
01(1.1)	INTRODUCTION TO PYTHON LIBRARIES FOR DATA SCIENCE	02-02-2026	1	
1.2	PANDAS	02-02-2026	5	
1.3	MATPLOTLIB	02-02-2026	7	
1.4	SCIKIT LEARN	09-02-2026	9	
02	PERFORM DATA EXPLORATION AND PRE PROCESSING IN PYTHON	14-02-2026	11	
03	IMPLEMENT REGULARIZED LINEAR REGRESSION	14-02-2026	13	
04	IMPLEMENT NAVIE BAYES CLASSIFIER	23-02-2026	14	
05	IMPLEMENT REGULARIZED LOGISTIC REGRESSION	02-03-2026	15	
06	BUILD MODULES USING DIFFERENT ENSEMBLING TECHNIQUES	09-03-2026	17	

07	BUILD MODELS USING DECESION TREES	09-03-2026	18	
08	BUILD THE MODELS USING DIFFERENT KERNAL FUNCTIONS	16-03-2026	19	
09	IMPLEMENT K-NEAREST NEIGHBOURS (KNN)CLASSIFICATION	30-03-2026	20	
10	K-MEAN CLUSTERING WITH PCA AND ELBOW METHOD	30-03-2026	21	

INTRODUCTION TO PYTHON LIBRARIES FOR DATA SCIENCE

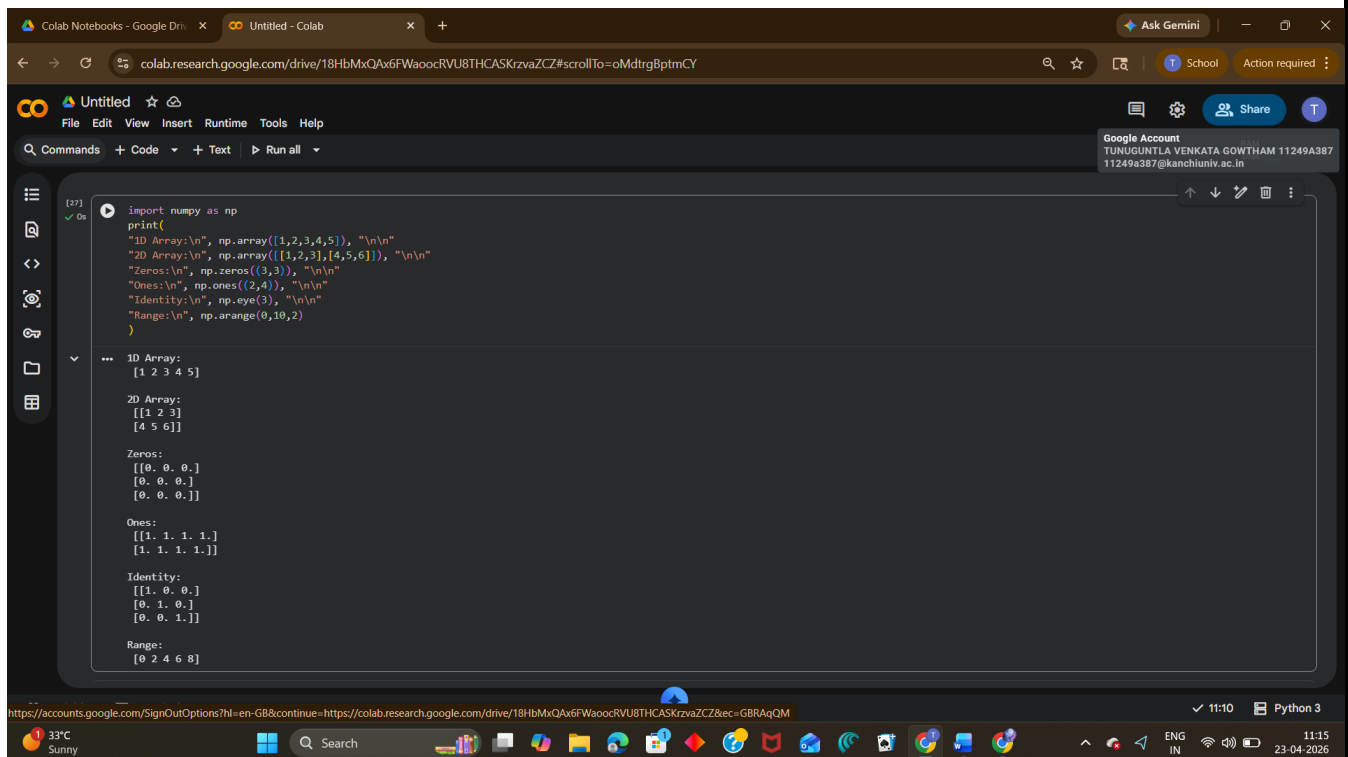
Exp-01(1.1)

Date-02-02-2026

Aim:

To learn basic Python data science libraries like NumPy and Pandas.

Program with output:



The screenshot shows a Google Colab notebook interface. The code cell contains the following Python code:

```
import numpy as np
print(
    "1D Array:\n", np.array([1,2,3,4,5]), "\n\n"
    "2D Array:\n", np.array([[1,2,3],[4,5,6]]), "\n\n"
    "Zeros:\n", np.zeros((3,3)), "\n\n"
    "Ones:\n", np.ones((2,4)), "\n\n"
    "Identity:\n", np.eye(3), "\n\n"
    "Range:\n", np.arange(0,10,2)
)
```

The output of the code is displayed below the code cell:

```
1D Array:
[1 2 3 4 5]

2D Array:
[[1 2 3]
 [4 5 6]]

Zeros:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

Ones:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]

Identity:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Range:
[0 2 4 6 8]
```

The notebook interface includes a top bar with the Google Colab logo, a file explorer on the left, and a bottom status bar showing the temperature (33°C), time (11:15), and date (23-04-2026).

Colab Notebooks - Google Drive x Untitled - Colab x + Ask Gemini

colab.research.google.com/drive/18HbMxQx6FWaocRVU8THCASKrzvaCZ#scrollTo=oMdtgBptmCY

Untitled

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Google Account
TUNICUNTLA VENKATA
GOWTHAM 11249A387
11249a387@kanchiuniv.ac.in

```
[7] ✓ Os
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(
    "Array a:", a, "\n"
    "Array b:", b, "\n\n"
    "Addition (a + b):", a + b, "\n"
    "Multiplication (a * b):", a * b, "\n"
    "ScalarMultiplication (a * 10):", a * 10, "\n\n"
    "Square Root of a:", np.sqrt(a), "\n"
    "Mean of a:", np.mean(a), "\n"
    "Dot Product (a · b):", np.dot(a, b)
)
```

Array a: [1 2 3]
Array b: [4 5 6]

Addition (a + b): [5 7 9]
Multiplication (a * b): [4 10 18]
ScalarMultiplication (a * 10): [10 20 30]

Square Root of a: [1. 1.41421356 1.73205081]
Mean of a: 2.0
Dot Product (a · b): 32

[] Start coding or generate with AI.

[8] import numpy as np

Variables Terminal

11:10 Python 3

Colab Notebooks - Google Drive x Untitled - Colab x + Ask Gemini

colab.research.google.com/drive/18HbMxQx6FWaocRVU8THCASKrzvaCZ#scrollTo=oMdtgBptmCY

Untitled

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Google Account
TUNICUNTLA VENKATA GOWTHAM 11249A387
11249a387@kanchiuniv.ac.in

```
[7] ✓ Os
import numpy as np

# 1. Array Creation
a = np.arange(1, 5, 1, dtype=int)
b = np.random.randn(3, 2)
c = np.linspace(1, 10, 5)
d = np.zeros(10)
e = np.ones(5, dtype=float)
f = np.empty(3)
g = np.random.randint(1, 100, size=(4, 3))

print("Array a:", a)
print("Range b:", b)
print("Linspace c:", c)
print("Zeros d:", d)
print("Ones e:", e)
print("Empty f:", f)
print("Random g:", g)

# 2. Array Properties
print("Shape:", g.shape)
print("Dimensions:", g.ndim)
print("Type:", g.dtype)
print("Data type:", g.dtype)

# 3. Reshape & Transpose
reshaped = g.reshape(12)
print("Reshaped array:", reshaped)
print("Dimensions:", g.shape)
print("Transposed:", g.T)

# 4. Mathematical Operations
print("Sum:", np.sum(a))
print("Product:", np.prod(a))
print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Variance:", np.var(a))
print("Standard Deviation:", np.std(a))
print("Max:", np.max(a))
print("Min:", np.min(a))
print("Absolute:", np.absolute(-a, 2, 3))

# 5. Statistical Functions
print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Standard Deviation:", np.std(a))
print("Variance:", np.var(a))
print("Max:", np.max(a))
print("Min:", np.min(a))
print("Percentile (50):", np.percentile(a, 50))

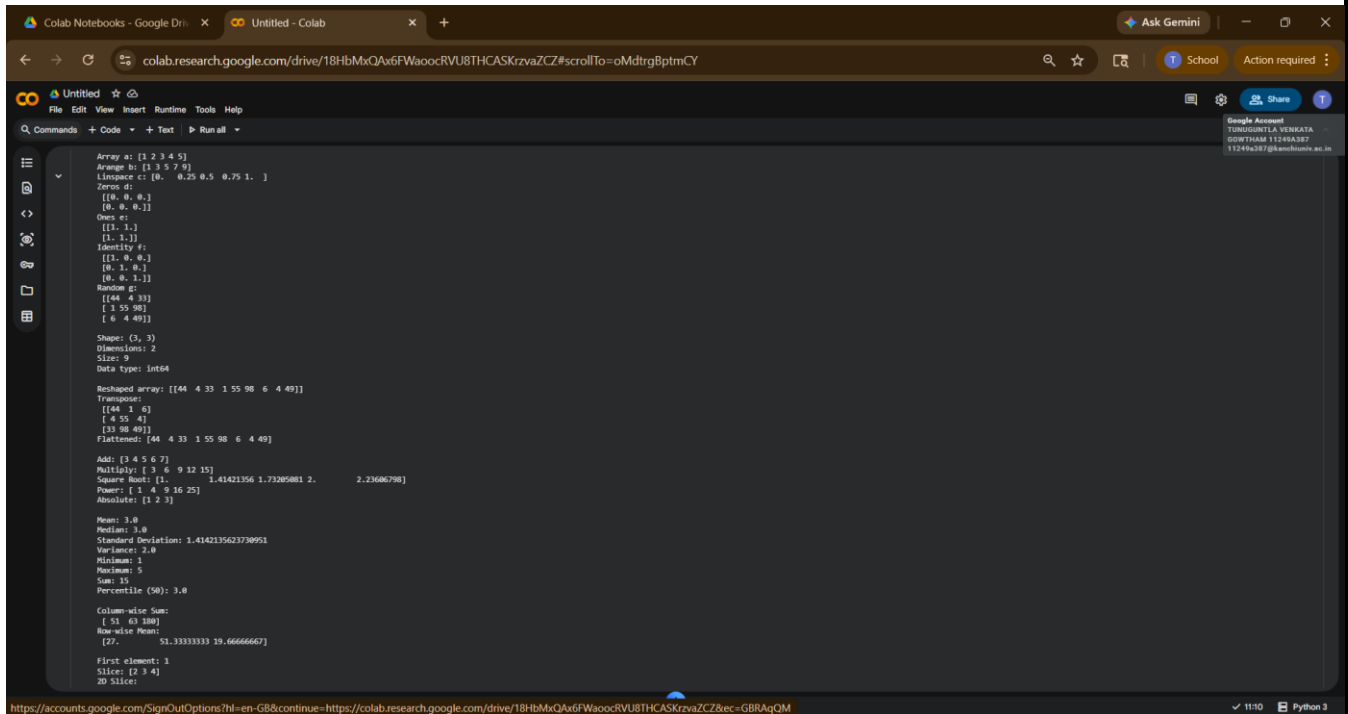
# 6. Axis-based operations
print("Sum along axis 0:", np.sum(a, axis=0))
print("Max along axis 0:", np.max(a, axis=0))

# 7. Indexing & Slicing
print("Slice (0:5):", a[0:5])
print("Slice (2:4):", a[2:4])
print("All slices:", a[:, 0:2])

# 8. Boolean & Conditional
print("Where (a > 5):", np.where(a > 5))
print("Where (a > 5):", np.where(a > 5))
print("All (a > 5):", np.all(a > 5))

# 9. Sorting & Searching
print("Sorted a:", np.sort(a))
print("Index of max:", np.argmax(a))
print("Index of min:", np.argmin(a))
```

OUTPUT-



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: `colab.research.google.com/drive/18HbMxQAv6FWaoocRVU8THCASKrvaZCZ#scrollTo=oMdtgBptmCY`. The notebook is titled "Untitled" and shows a single code cell with the following Python code and its output:

```
Array a: [1 2 3 4 5]
Average a: [3.3 5.7 9]
Linspace c: [0. 0.25 0.5 0.75 1. ]
Zeros d:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
Ones e:
[[1. 1.]
 [1. 1.]
 [1. 1.]]
Identity f:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Random g:
[[44. 4 33]
 [ 1 55 98]
 [ 6 4 49]]

Shape: (3, 3)
Dimensions: 2
Slice: 9
Data type: int64

Reshaped array: [[44. 4 33. 1 55 98. 6 4 49]]
Transposed:
[[44. 1 4]
 [ 4 55 4]
 [33 98 49]]
Flattened: [44. 4 33. 1 55 98. 6 4 49]

Add: [3 4 5 6 7]
Multiply: [ 3. 6. 9 12 15]
Square Root: [1. 1.41421356 1.73205081 2. 2.23606798]
Power: [ 1. 4. 9 16 25]
Absolute: [1 2 3]

Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623730951
Variance: 2.0
Minimum: 1
Maximum: 5
Sum: 15
Percentile (50): 3.0

Column-wise Sum:
[ 51. 63 100]
Row-wise Mean:
[27. 51.33333333 19.66666667]

First element: 1
Slice: [2 3 4]
2D Slice:
```

Result:

Thus, it is implemented Introduct on to Python Libraries for Data Science successfully.

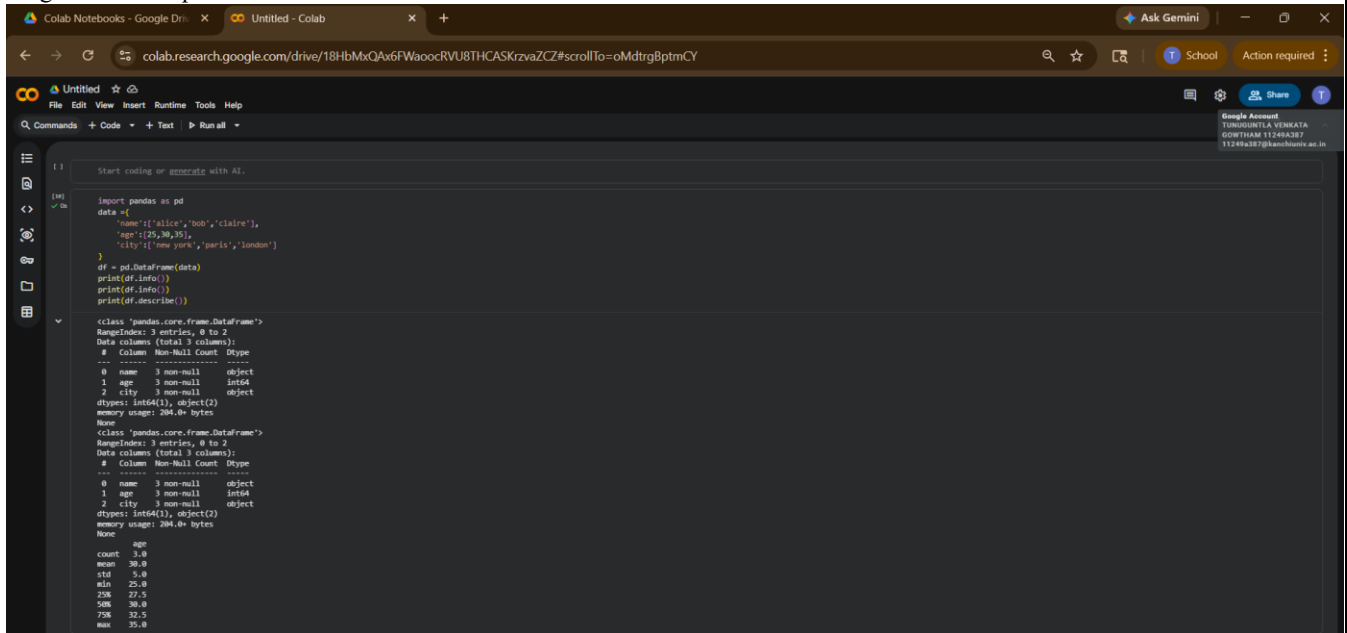
PANDAS

EXP-1.2 DATE-02-02-2026

Aim:

To study the Pandas library for data handling and analysis.

Program with output:



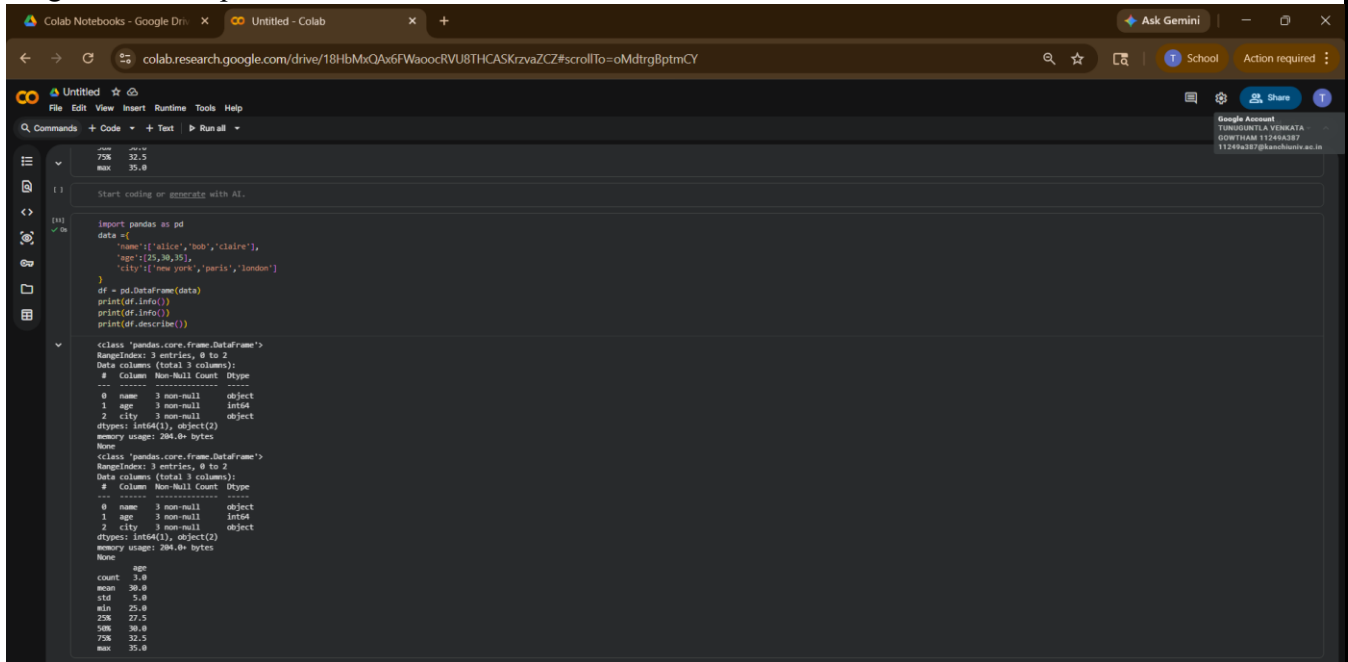
The screenshot shows a Google Colab notebook interface. The top bar includes the Google Colab logo, the text "Colab Notebooks - Google Drive", and a tab titled "Untitled - Colab". The address bar shows the URL "colab.research.google.com/drive/18HbMxQAx6FWaoocRVU8THCASKrzvaZCZ#scrollTo=oMdtrg8ptmCY". The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with icons for commands, code, text, and run. The main code cell contains the following Python code:

```
[14] import pandas as pd
data = {
    'name': ['alice', 'bob', 'claire'],
    'age': [25, 30, 35],
    'city': ['new york', 'paris', 'london']
}
df = pd.DataFrame(data)
print(df.info())
print(df.info())
print(df.describe())
```

The output of the code is displayed below the code cell. It shows the class of the DataFrame, the number of entries, the data columns, the dtypes, and the memory usage. The output is as follows:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  --
 0   name    3 non-null        object
 1   age     3 non-null        int64
 2   city    3 non-null        object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  --
 0   name    3 non-null        object
 1   age     3 non-null        int64
 2   city    3 non-null        object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
count      3.0
mean      30.0
std        5.0
min       25.0
25%       27.5
50%       30.0
75%       32.5
max       35.0
```


Program with output:



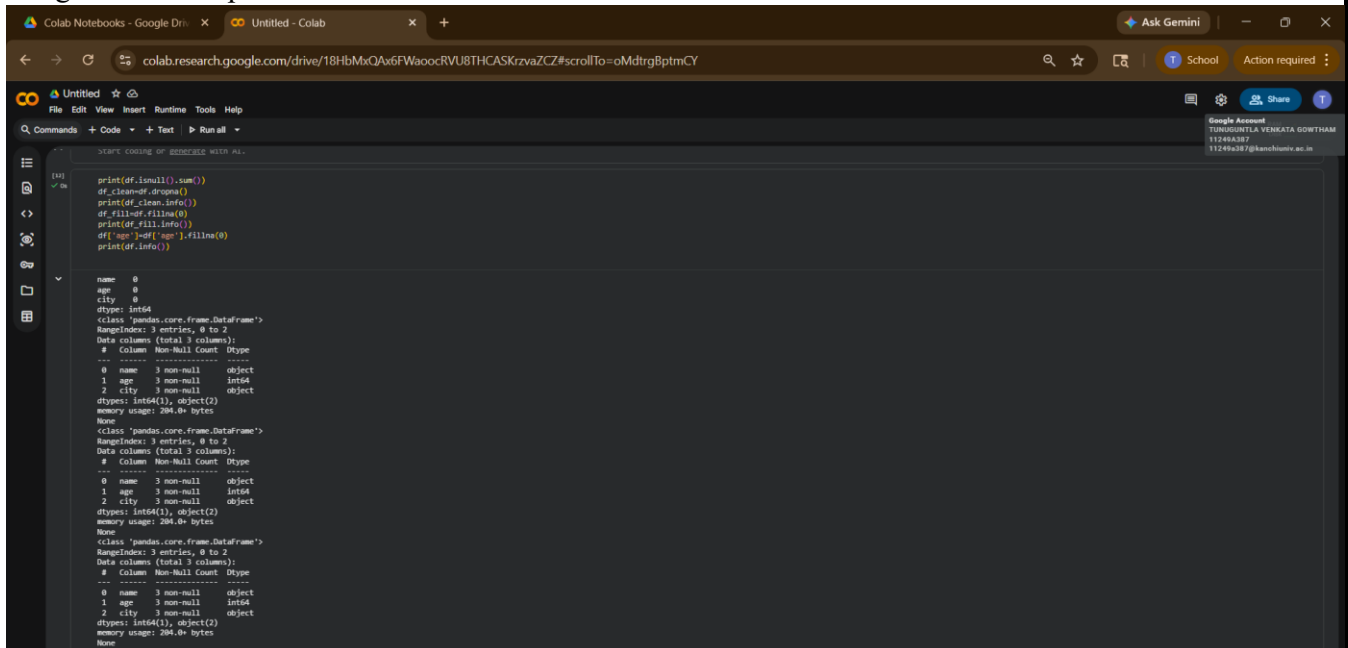
The screenshot shows a Google Colab notebook with the following code and output:

```
import pandas as pd
data = {
    'name': ['Alice', 'Bob', 'Claire'],
    'age': [25, 30, 35],
    'city': ['New York', 'Paris', 'London']
}
df = pd.DataFrame(data)
print(df.info())
print(df.describe())
```

The output displays the DataFrame's structure and summary statistics:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  --
 0   name    3 non-null    object
 1   age     3 non-null    int64
 2   city    3 non-null    object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  --
 0   name    3 non-null    object
 1   age     3 non-null    int64
 2   city    3 non-null    object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
count      3
age        3
mean       30.0
std        5.0
min        25.0
25%        27.5
50%        30.0
75%        32.5
max        35.0
```

Program with output:



The screenshot shows a Google Colab notebook with the following code and output:

```
print(df.isnull().sum())
df_clean = df.dropna()
print(df_clean.info())
df_fill = df.fillna(0)
print(df_fill.info())
df['age'] = df['age'].fillna(0)
print(df.info())
```

The output displays the DataFrame's structure and summary statistics after cleaning:

```
name      0
age        0
city       0
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  --
 0   name    3 non-null    object
 1   age     3 non-null    int64
 2   city    3 non-null    object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  --
 0   name    3 non-null    object
 1   age     3 non-null    int64
 2   city    3 non-null    object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
```

Result:

Thus, it is implemented Pandas successfully.

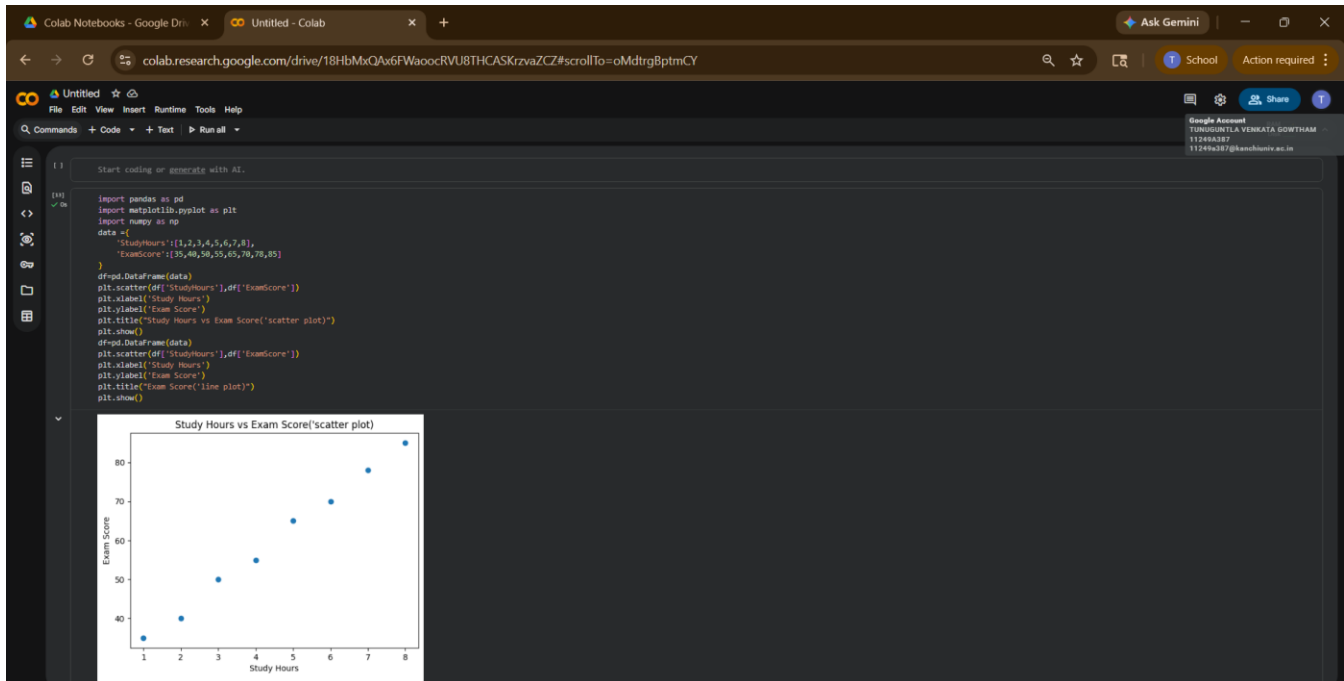
MATPLOTLIB

EXP-1.3 DATE-02-02-2026

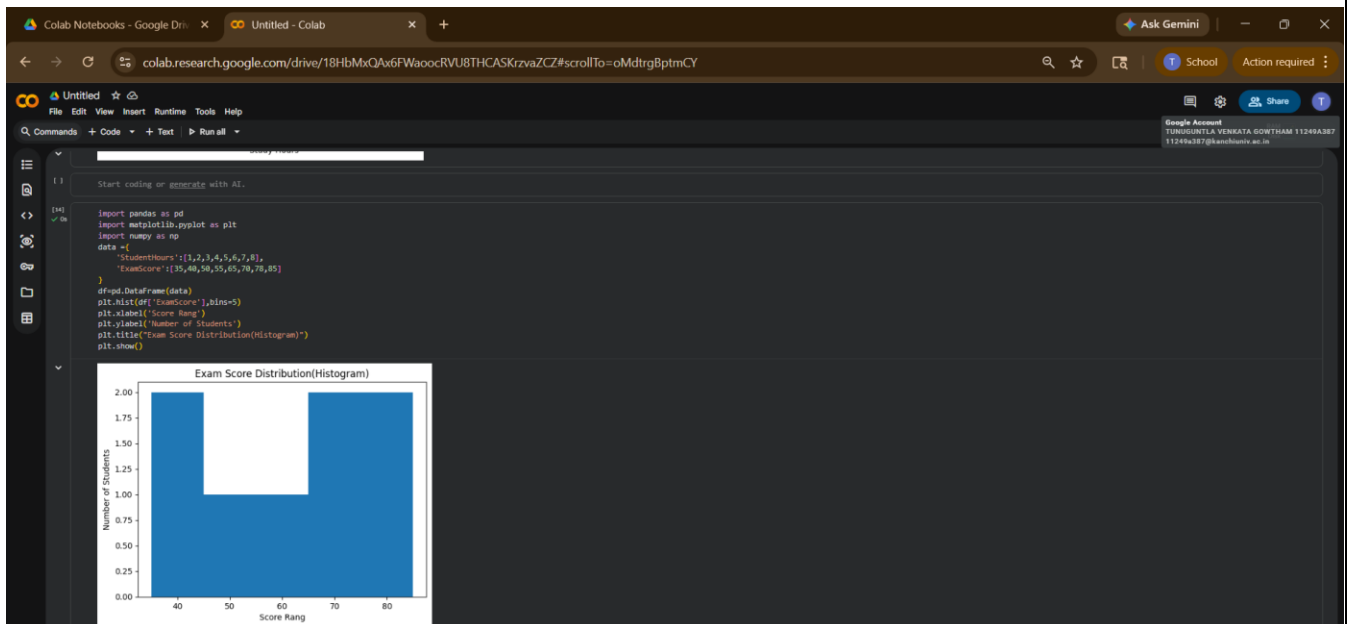
Aim:

To study the Matplotlib library for data visualization.

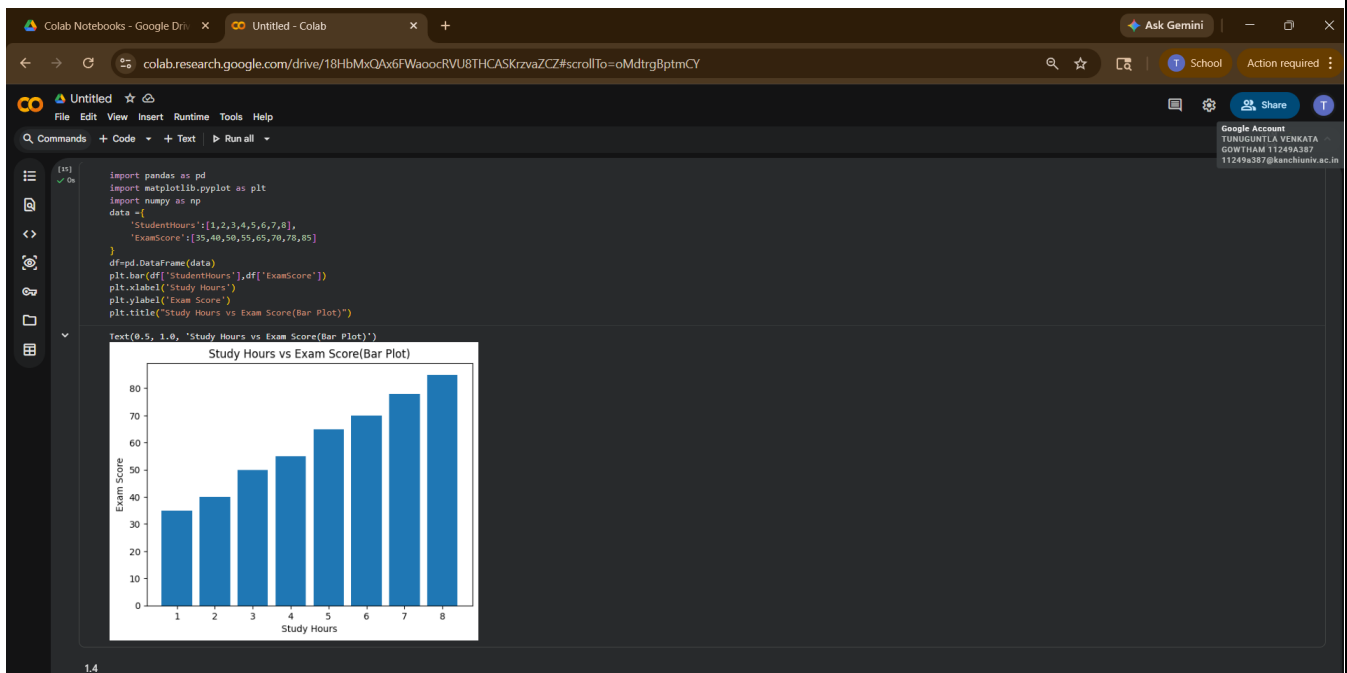
PROGRAM WITH OUTPUT-



PROGRAM WITH OUTPUT-



PROGRAM WITH OUTPUT-



Result:

Thus, it is implemented Matplotlib successfully.

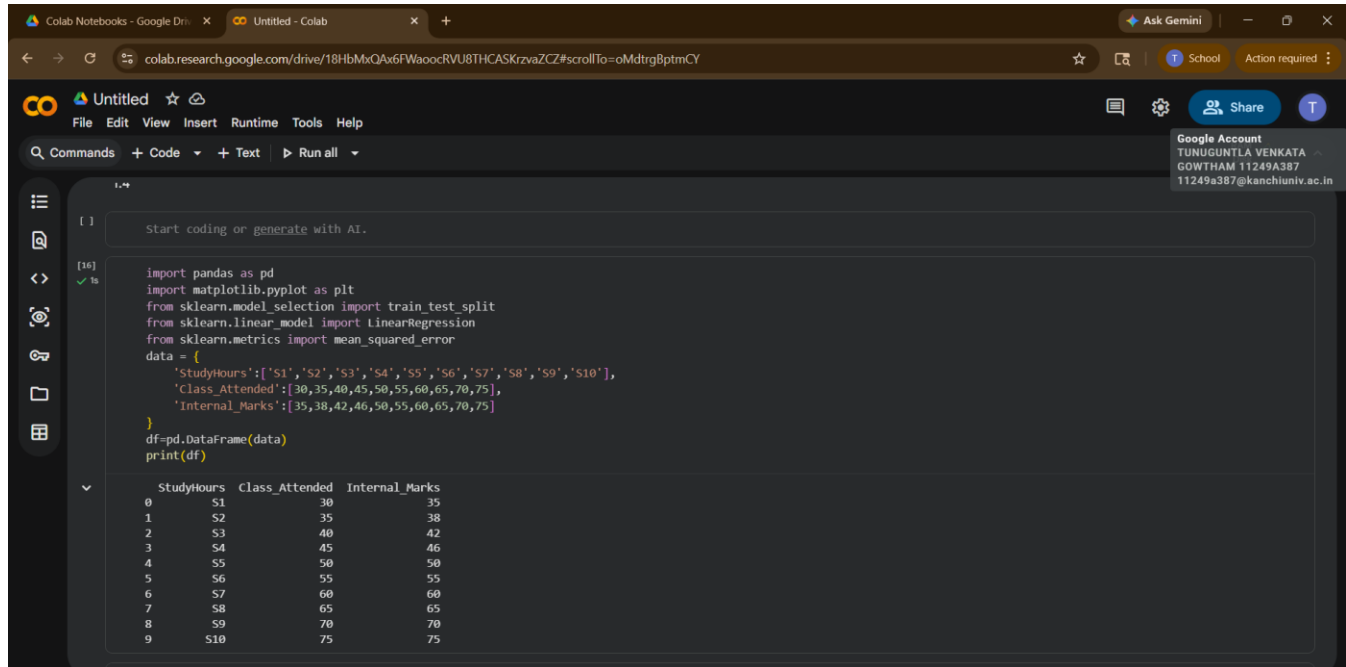
SCIKIT LEARN

EXP N-1.4 DATE-09-02-2026

Aim:

To study the scikit-learn library for machine learning.

PROGRAM WITH OUTPUT-



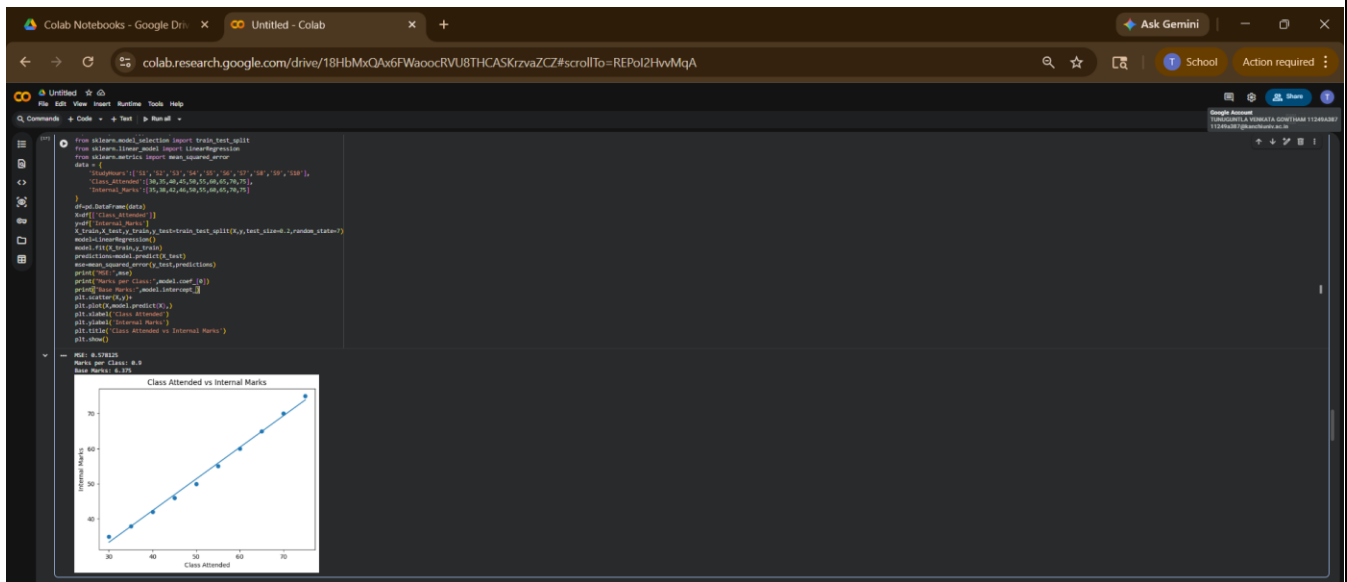
The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, the name 'Untitled', and a 'Share' button. The left sidebar contains icons for file management, code execution, and output. The main area displays a Python code cell with the following code:

```
[16] ✓ ts
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
data = {
    'StudyHours': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10'],
    'Class_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
```

The output of the code is a DataFrame with the following data:

	StudyHours	Class_Attended	Internal_Marks
0	S1	30	35
1	S2	35	38
2	S3	40	42
3	S4	45	46
4	S5	50	50
5	S6	55	55
6	S7	60	60
7	S8	65	65
8	S9	70	70
9	S10	75	75

PROGRAM WITH OUTPUT-



Result:

Thus, it is implemented Scikit-learn successfully.

PERFORM DATA EXPLORATION AND PREPROCESSING IN PYTHON

EXP-02

DATE-14-02-2026

Aim:

To perform data exploration and preprocessing using Python.

PROGRAM :

```
Colab Notebooks - Google Drive x Untitled - Colab x + Ask Gemini
colab.research.google.com/drive/18HbMxQx6FWaocRVU8THCASKrZvaZCZ#scrollTo=REPol2HwMqA

File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = {
    'country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],
    'Age': [22, 25, np.nan, 30, 28, 35],
    'salary': [400000, 60000, 50000, np.nan, 72000, 58000],
    'purchased': ['no', 'yes', 'yes', 'no', 'yes', 'yes']
}

df = pd.DataFrame(data)
print('---ORIGINAL RAW DATA---')
print(df)
print('\n')
print('---MISSING VALUE COUNT---')
print(df.isnull().sum())
print('\n')
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['salary'] = df['salary'].fillna(df['salary'].mean())
print('---DATA AFTER CLEANING---')
print(df)
print('\n')
df_encoded = pd.get_dummies(df, columns=['country'])
df_encoded['Purchased'] = df_encoded['purchased'].map({'yes': 1, 'no': 0})
print('---DATA AFTER ENCODING(All Number Now---')
print(df_encoded)
print('\n')
df_cleaned = df.copy()
df_cleaned['Age'] = df_cleaned['Age'].fillna(df_cleaned['Age'].mean())
df_cleaned['salary'] = df_cleaned['salary'].fillna(df_cleaned['salary'].mean())
df_encoded = pd.get_dummies(df_cleaned, columns=['country'])
df_encoded['Purchased'] = df_encoded['purchased'].map({'yes': 1, 'no': 0})

x = df_encoded.drop(['Purchased', 'purchased'], axis=1)
y = df_encoded['Purchased']

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
print('---FINAL PROCESSED DATA(Ready for AI---')
print(pd.DataFrame(x_scaled, columns=x.columns).head())
print('\n')
```

Output:

```
Colab Notebooks - Google Drive x Untitled - Colab x + Ask Gemini
colab.research.google.com/drive/18HbMxQx6FWaocRVU8THCASKrZvaZCZ#scrollTo=REPol2HwMqA

File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

---ORIGINAL RAW DATA---
country  Age  salary purchased
0  India  22.0  400000.0      no
1  USA    25.0   60000.0     yes
2  India  NaN    50000.0     yes
3  USA    28.0   NaN      no
4  UK     28.0   72000.0     yes
5  India  35.0   58000.0     yes

---MISSING VALUE COUNT---
country    0
Age        1
salary     1
purchased  0
dtype: int64

---DATA AFTER CLEANING---
country  Age  salary purchased
0  India  22.0  400000.0      no
1  USA    25.0   60000.0     yes
2  India  25.0   50000.0     yes
3  USA    30.0  120000.0     no
4  UK     28.0   72000.0     yes
5  India  35.0   58000.0     yes

---DATA AFTER ENCODING(All Number Now---
Age  salary purchased  country_UK  country_USA  country_India  Purchased
0  22.0  400000.0      no      False      False      True         0
1  25.0  60000.0     yes      False      True       False         1
2  25.0  50000.0     yes      False      False      True         1
3  30.0  120000.0     no      False      True       False         0
4  28.0  72000.0     yes      True       False      False         1
5  35.0  58000.0     yes      False      False      True         1

---FINAL PROCESSED DATA(Ready for AI---
Age  salary  country_UK  country_USA  country_India
0 -1.404615  2.107920  -0.447214  -0.707107         1.0
1 -0.262187 -0.546090  -0.447214  1.414214        -1.0
2  0.000000 -0.627429  -0.447214  -0.707107         1.0
3  0.404672  0.000000  -0.447214  1.414214        -1.0
4  0.000000 -0.404672  1.106062  -0.707107        -1.0
```

Result:

The dataset was explored, cleaned, and pre processed successfully, making it ready for further analysis or model building.

IMPLEMENT REGULARIZED LINEAR REGRESSION

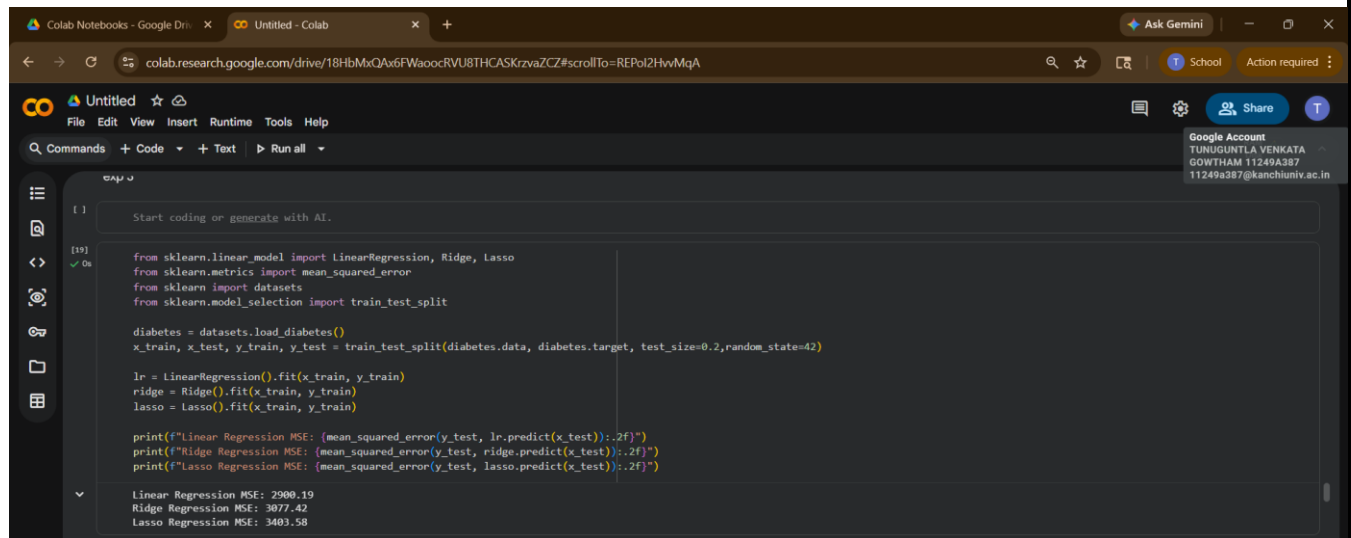
EXP-03

DATE-14-02-2026

Aim

To implement Regularized Linear Regression using Python.

PROGRAM WITH OUTPUT



```
[1] Start coding or generate with AI.

[19] ✓ 0s
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn import datasets
from sklearn.model_selection import train_test_split

diabetes = datasets.load_diabetes()
x_train, x_test, y_train, y_test = train_test_split(diabetes.data, diabetes.target, test_size=0.2, random_state=42)

lr = LinearRegression().fit(x_train, y_train)
ridge = Ridge().fit(x_train, y_train)
lasso = Lasso().fit(x_train, y_train)

print(f"Linear Regression MSE: {mean_squared_error(y_test, lr.predict(x_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test, ridge.predict(x_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test, lasso.predict(x_test)):.2f}")

Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 3403.58
```

Result

The regularized linear regression model was successfully implemented using Python. The model reduced overfitting and improved prediction performance by applying regularization techniques such as Ridge or Lasso regression.

IMPLEMENT NAVIE BAYES CLASSIFIER

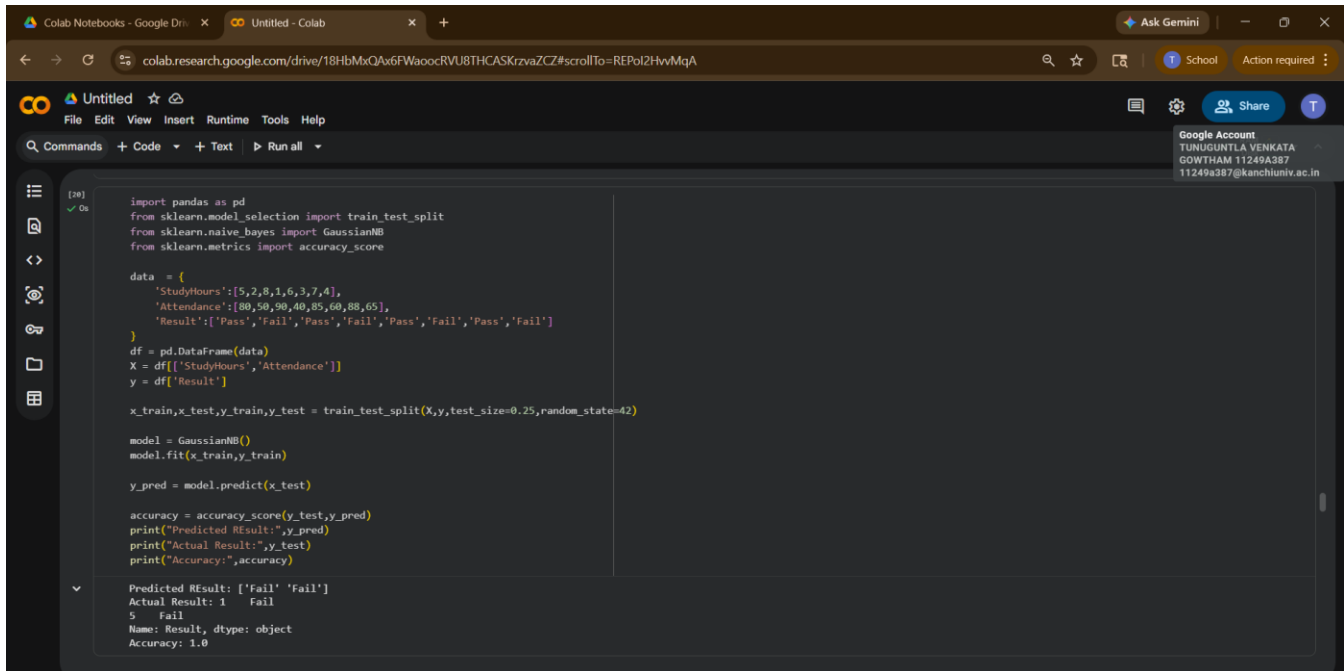
EXP-04

DATE-23-02-2026

Aim

To implement a Naive Bayes classifier using Python for classification of data.

PROGRAM WITH OUTPUT



```
[28] ✓ Qs
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = {
    'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
}
df = pd.DataFrame(data)
X = df[['StudyHours', 'Attendance']]
y = df['Result']

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

model = GaussianNB()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

accuracy = accuracy_score(y_test, y_pred)
print("Predicted Result:", y_pred)
print("Actual Result:", y_test)
print("Accuracy:", accuracy)

Predicted Result: ['Fail' 'Fail']
Actual Result: 1    Fail
5    Fail
Name: Result, dtype: object
Accuracy: 1.0
```

Result

The Naive Bayes classifier was successfully implemented using Python. The model was trained and used to classify the data effectively, providing accurate predictions based on probability.

IMPLEMENT REGULARIZED LOGISTIC REGRESSION

EXP-05

DATE-02-03-2026

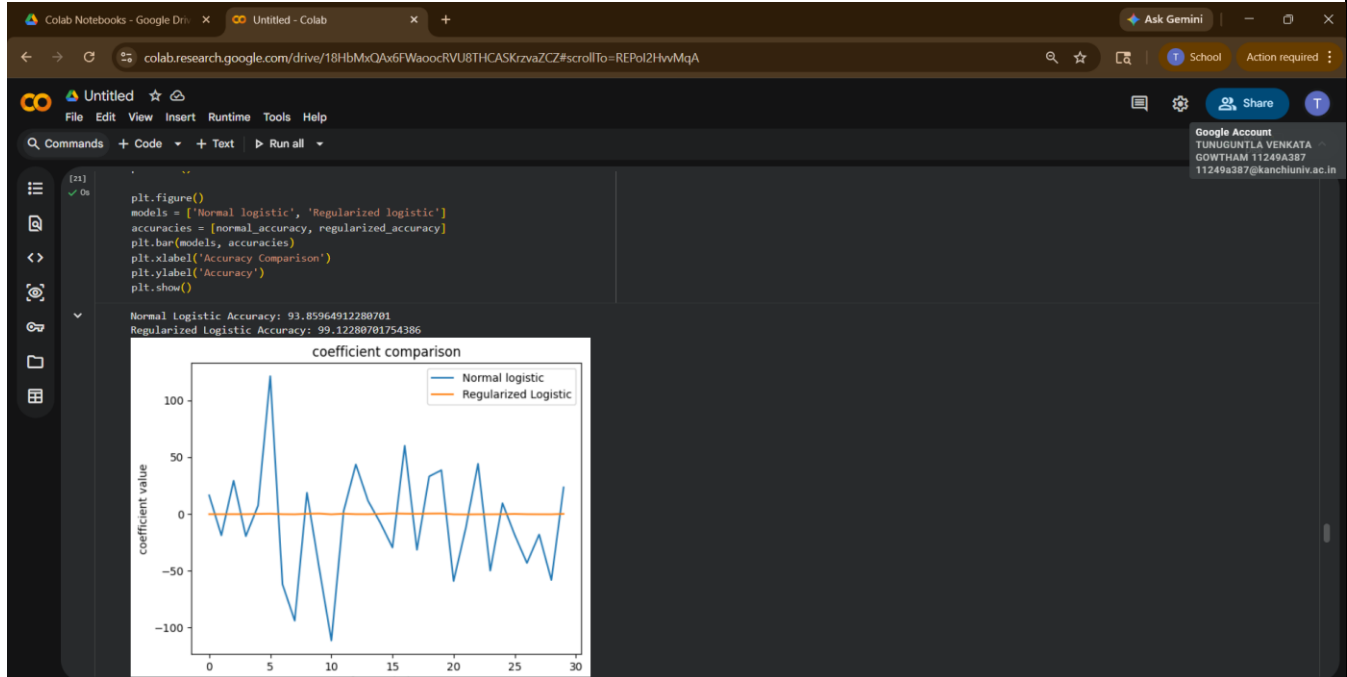
Aim:

To implement Regularized Logistic Regression using Python.

Program:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.metrics import accuracy_score
8
9 data = load_breast_cancer()
10 x = data.data
11 y = data.target
12
13 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
14
15 scaler = StandardScaler()
16 x_train_scaled = scaler.fit_transform(x_train)
17 x_test_scaled = scaler.transform(x_test)
18
19 normal_model = LogisticRegression(C=0.1, penalty='l1', solver='liblinear')
20 normal_model.fit(x_train_scaled, y_train)
21 normal_pred = normal_model.predict(x_test_scaled)
22 normal_accuracy = accuracy_score(y_test, normal_pred)
23
24 regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
25 regularized_model.fit(x_train_scaled, y_train)
26 regularized_pred = regularized_model.predict(x_test_scaled)
27 regularized_accuracy = accuracy_score(y_test, regularized_pred)
28
29 print('Normal Logistic Accuracy:', normal_accuracy * 100)
30 print('Regularized Logistic Accuracy:', regularized_accuracy * 100)
31
32 plt.figure()
33 plt.plot(normal_model.coef_.flatten(), label='Normal Logistic')
34 plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic')
35 plt.legend()
36 plt.title('Coefficient Comparison')
37 plt.xlabel('Coefficient Index')
38 plt.ylabel('Coefficient Value')
39 plt.show()
40
41 plt.figure()
42 models = ['Normal Logistic', 'Regularized Logistic']
43 accuracies = [normal_accuracy, regularized_accuracy]
44 plt.bar(models, accuracies)
45 plt.xlabel('Accuracy Comparison')
46 plt.ylabel('Accuracy')
47 plt.show()
```

Output:



Result

The regularized logistic regression model was successfully implemented using Python. Regularization helped reduce overfitting and improved the model's classification performance and generalization on the dataset.

BUILD MODULES USING DIFFERENT ENSEMBLING TECHNIQUES

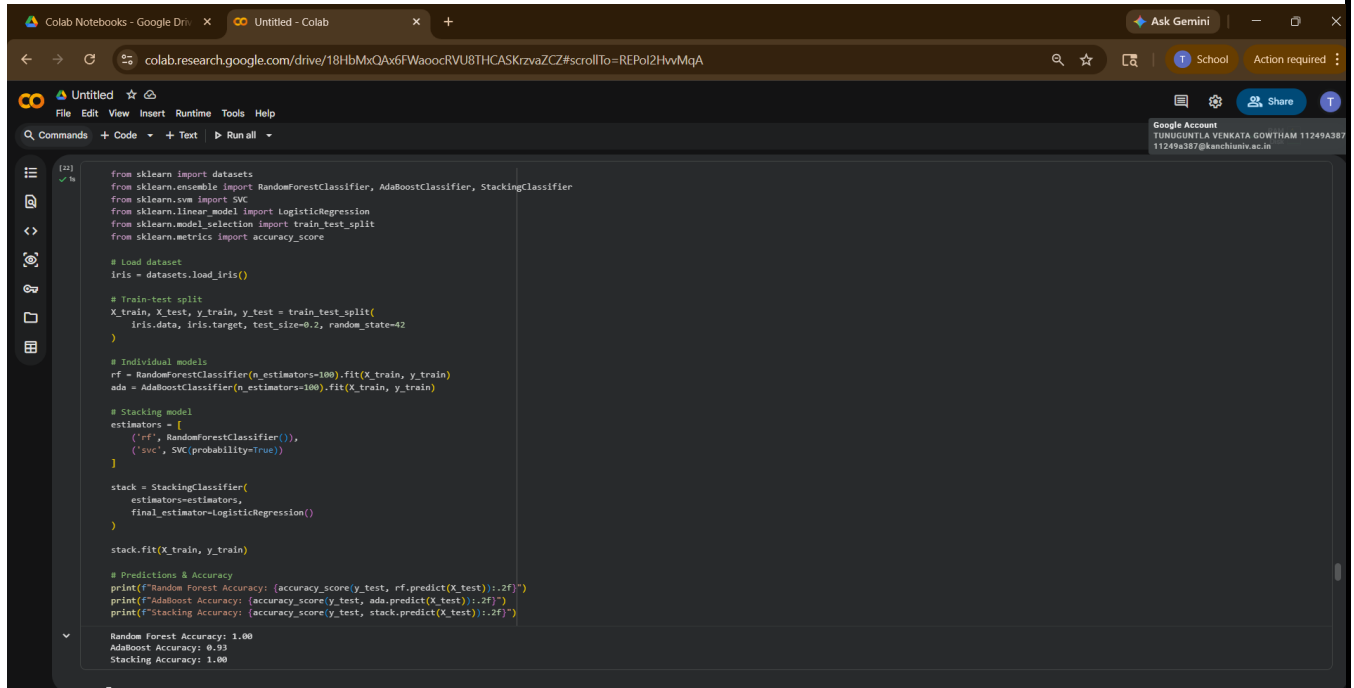
EXP-06

DATE-09-03-2026

Aim:

To build models using different ensemble techniques in Python.

PROGRAM WITH OUTPUT



```
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = datasets.load_iris()

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# Individual models
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)

# Stacking model
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)

stack.fit(X_train, y_train)

# Predictions & Accuracy
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")

Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

Result

The ensemble models were successfully built using techniques such as Bagging, Boosting, and Random Forest. These methods improved prediction accuracy and reduced overfitting compared to individual models.

BUILD MODELS USING DECISION TREES

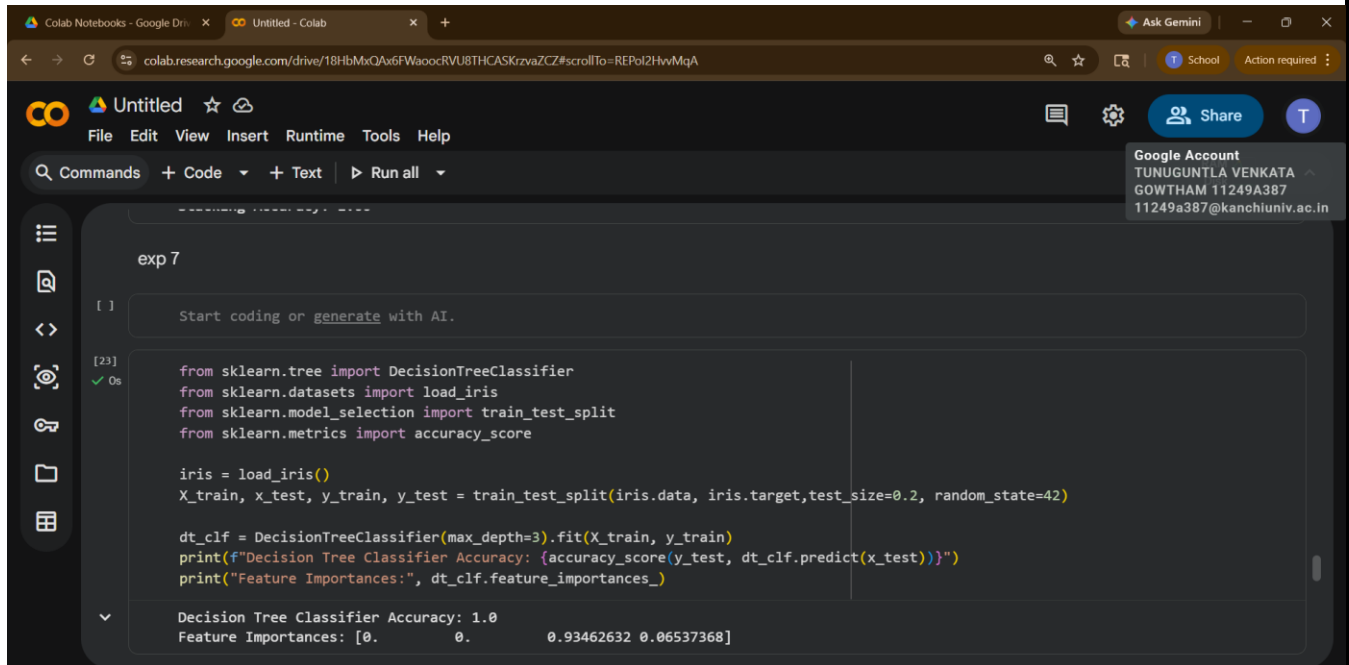
EXP-7

DATE-09-03-2026

Aim:

To build a classification model using Decision Trees in Python.

PROGRAM WITH OUTPUT



```
exp 7

[ ] Start coding or generate with AI.

[23] ✓ 0s
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2, random_state=42)

dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, dt_clf.predict(x_test))}")
print("Feature Importances:", dt_clf.feature_importances_)

Decision Tree Classifier Accuracy: 1.0
Feature Importances: [0. 0. 0.93462632 0.06537368]
```

Result:

The Decision Tree model was successfully implemented using Python. The model learned decision rules from the dataset and provided accurate predictions for classification tasks.

BUILD THE MODELS USING DIFFERENT KERNEL FUNCTIONS

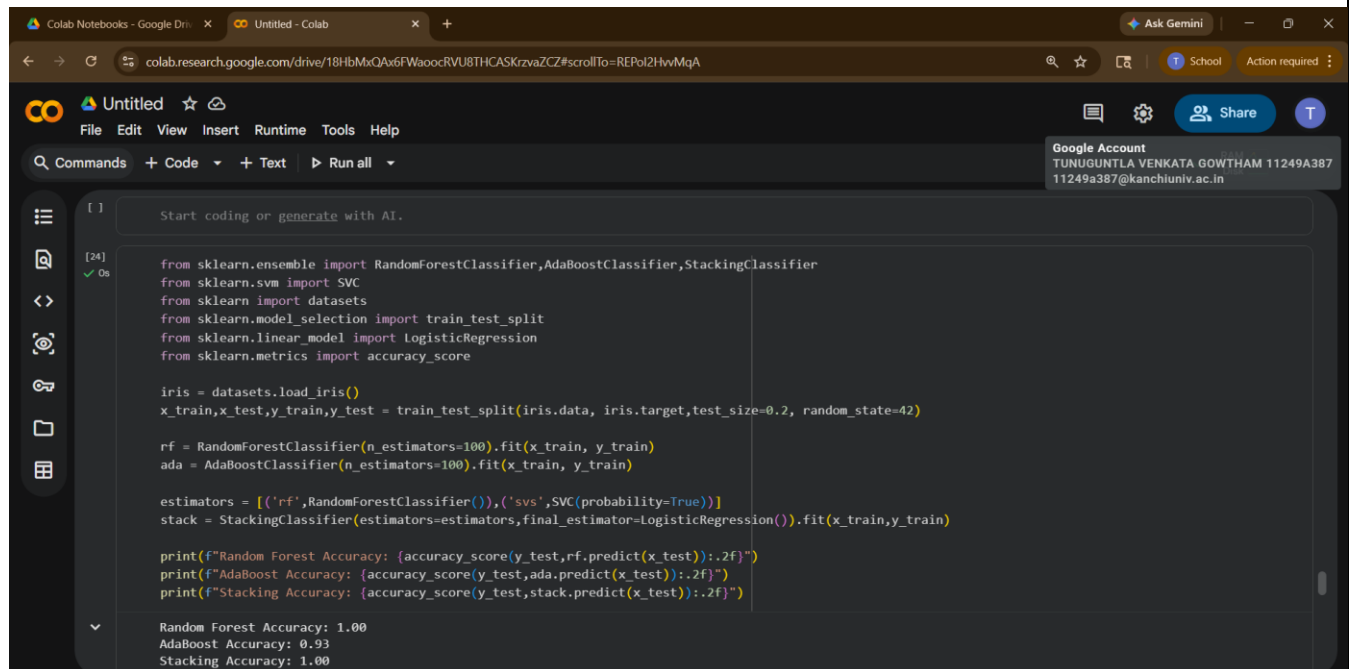
EXP-08

DATE-16-03-2026

Aim

To build models using different kernel functions in Python.

PROGRAM WITH OUTPUT



The screenshot shows a Google Colab notebook interface. The top bar includes the Colab logo, the title 'Untitled - Colab', and a 'Share' button. The main area contains a code cell with the following Python code:

```
[24] from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
      from sklearn.svm import SVC
      from sklearn import datasets
      from sklearn.model_selection import train_test_split
      from sklearn.linear_model import LogisticRegression
      from sklearn.metrics import accuracy_score

      iris = datasets.load_iris()
      x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2, random_state=42)

      rf = RandomForestClassifier(n_estimators=100).fit(x_train, y_train)
      ada = AdaBoostClassifier(n_estimators=100).fit(x_train, y_train)

      estimators = [('rf', RandomForestClassifier()), ('svm', SVC(probability=True))]
      stack = StackingClassifier(estimators=estimators, final_estimator=LogisticRegression()).fit(x_train, y_train)

      print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(x_test)):.2f}")
      print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(x_test)):.2f}")
      print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(x_test)):.2f}")
```

The output of the code is displayed below the code cell:

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

Result

The model was successfully implemented using kernel-based methods (such as SVM). Different kernel functions like linear, polynomial, and RBF were used, and they improved classification performance based on the dataset structure.

IMPLEMENT K-NEAREST NEIGHBOURS (KNN) CLASSIFICATION

EXP-09 DATE-30-3-

2026 Aim:

To implement the K-Nearest Neighbours (KNN) classification algorithm using Python.

PROGRAM:

```
Colab Notebooks - Google Drive x Untitled - Colab x +
colab.research.google.com/drive/18HbMxQA6FWaocRVU8THCASKrzvaZCZ#scrollTo=REPol2HvMqA

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Create a small example dataset (similar to mobile dataset)
data = {
    'battery_power': [842, 1821, 563, 615, 1821, 1859, 1954, 1445],
    'ram': [2549, 2631, 2683, 2769, 3411, 1067, 275, 296],
    'px_height': [28, 965, 1263, 1216, 1288, 1884, 381, 512],
    'px_width': [758, 1884, 1736, 1786, 1212, 1654, 808, 1140],
    'price_range': [1, 2, 2, 2, 1, 1, 0, 0] # Target (0-3 in real dataset)
}

df = pd.DataFrame(data)

# Step 3: Features and target
X = df.drop(columns='price_range')
y = df['price_range']

# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=42, stratify=y
)

# Step 5: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 6: Create KNN model
knn = KNeighborsClassifier(n_neighbors=3)

# Step 7: Train model
knn.fit(X_train, y_train)

# Step 8: Prediction
y_pred = knn.predict(X_test)

# Step 9: Evaluation
print("Actual:", list(y_test))
print("Predicted:", list(y_pred))

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output:

```
Colab Notebooks - Google Drive x Untitled - Colab x +
colab.research.google.com/drive/18HbMxQA6FWaocRVU8THCASKrzvaZCZ#scrollTo=REPol2HvMqA

Actual: [2, 0, 1, 1]
Predicted: [np.int64(2), np.int64(0), np.int64(0), np.int64(0)]

Accuracy: 0.5

Confusion Matrix:
[[1 0 0]
 [2 0 0]
 [0 0 1]]

Classification Report:
      precision    recall  f1-score   support

0         0.33      1.00      0.50         1
1         0.00      0.00      0.00         2
2         1.00      1.00      1.00         1

 accuracy          0.50      0.50      0.50         4
 macro avg         0.44      0.67      0.50         4
 weighted avg         0.33      0.50      0.38         4

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this warning.
warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this warning.
warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this warning.
warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
```

Result

The KNN classification model was successfully implemented using Python. The model classified data based on the nearest neighbors and produced effective and accurate predictions for the given dataset.

K-MEAN CLUSTERING WITH PCA AND ELBOW METHOD

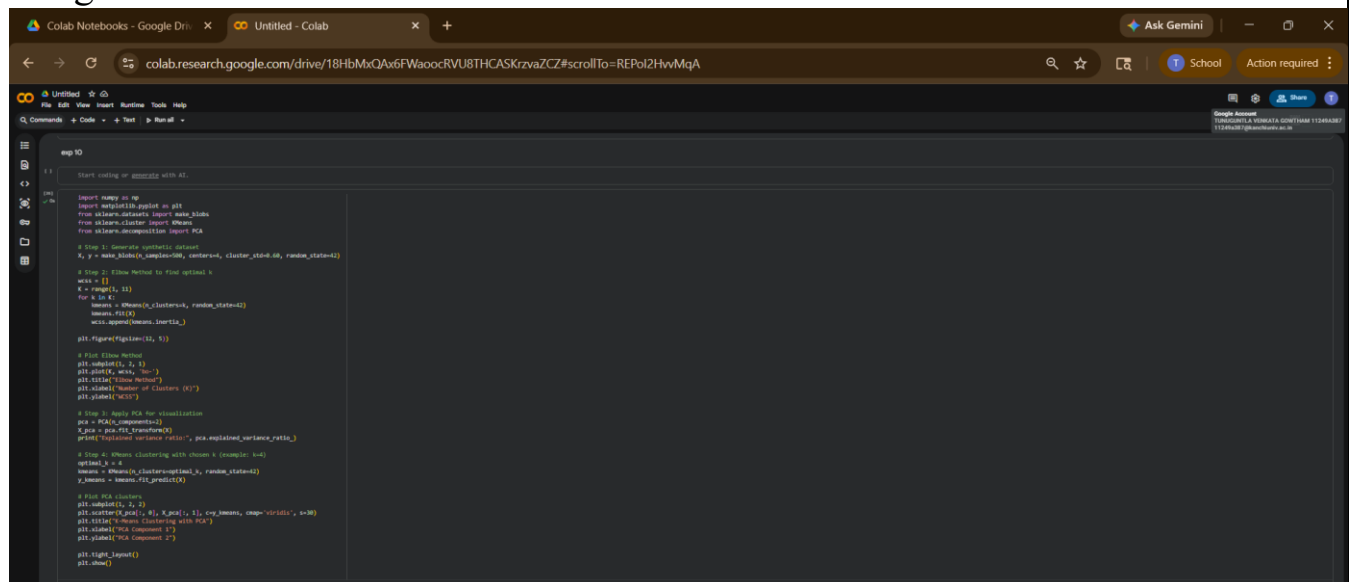
EXP-10

DATE-30-03-2026

Aim

To perform K-Means clustering using Python along with PCA and the Elbow method.

Program:



```
exp 10
Start coding or downloading with AI.

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.preprocessing import PCA

# Step 1: Generate synthetic dataset
X, y = make_blobs(n_samples=100, centers=4, cluster_std=0.5, random_state=42)

# Step 2: Elbow Method to find optimal k
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(12, 8))

# Plot Elbow Method
plt.subplot(2, 1, 1)
plt.plot(k, wcss, 'b-')
plt.title('Elbow Method')
plt.xlabel('Number of clusters (k)')
plt.ylabel('WCSS')

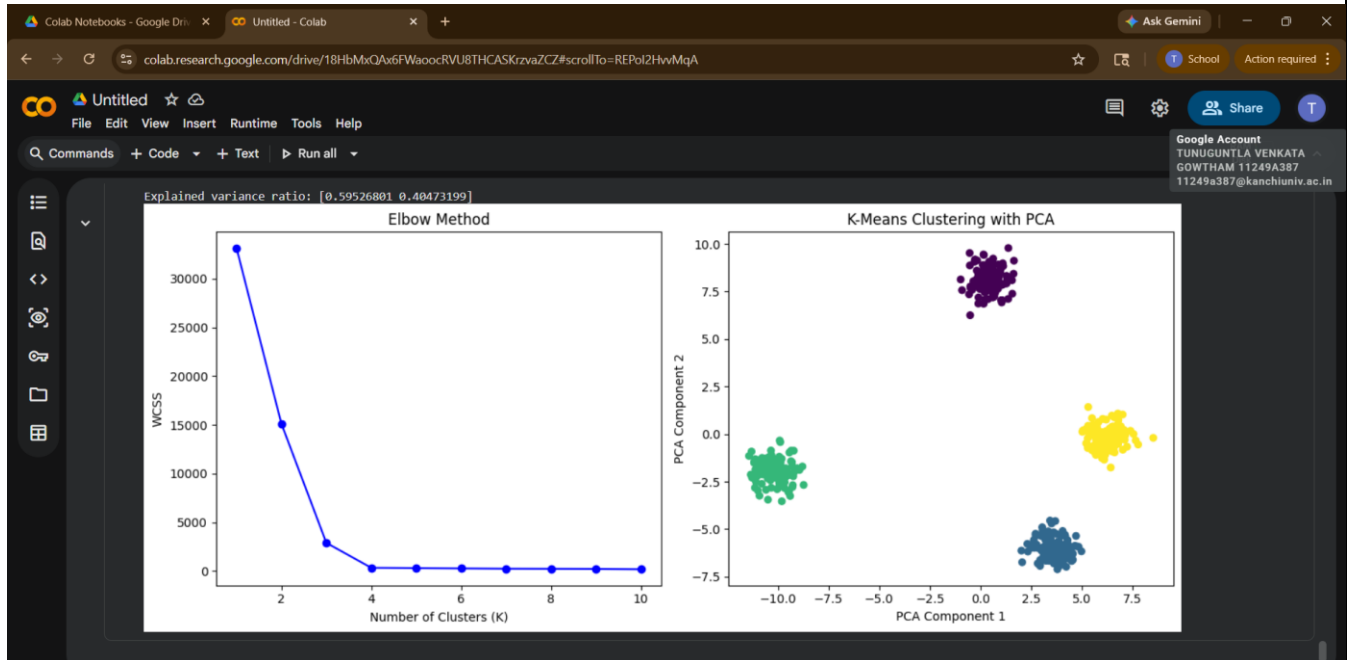
# Step 3: Apply PCA for visualization
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print('Explained variance ratio:', pca.explained_variance_ratio_)

# Step 4: K-Means clustering with chosen k (example: k=4)
optimal_k = 4
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
y_kmeans = kmeans.fit_predict(X)

# Plot PCA clusters
plt.subplot(2, 1, 2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_kmeans, cmap='viridis', s=50)
plt.title('K-Means Clustering with PCA')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')

plt.tight_layout()
plt.show()
```

OUTPUT:



Result

K-Means clustering was successfully implemented. The optimal number of clusters was identified using the Elbow method, and PCA was used to reduce dimensionality for better visualization. The data was grouped into meaningful clusters effectively.